Approcci neuro-simbolici al problema SAT

Intelligent Systems for Pattern Recognition Reinforcement Learning

Donato Meoli

Università di Pisa

Survey con progetto di codice

Il problema SAT

- SAT: data una formula booleana in forma normale congiuntiva $\varphi = \bigwedge_j (\bigvee_{\ell \in C_j} \ell)$, esiste un assegnamento delle variabili che ne soddisfa tutte le clausole? È stato il primo problema dimostrato NP-completo, ed è il motore di verifica, pianificazione e sintesi.
- I solutori moderni sono CDCL: *decidono* una variabile, *propagano*, e a ogni *conflitto* imparano una clausola e fanno backjump.
- L'euristica di branching (quale variabile decidere, di solito VSIDS) determina le prestazioni, ma all'inizio della ricerca i suoi contatori di attività dicono poco.
- Questo progetto: imparare l'euristica di branching con due metodi neuro-simbolici, portarli entrambi a uno stack PyTorch attuale, verificare il port sui checkpoint pubblicati, e misurare dove l'attenzione su grafo aiuta.

Percorso

Due modi di imparare un'euristica per SAT:

- Graph-Q-SAT: reinforcement learning basato sul valore (DQN) con una rete su grafo sul grafo della formula, che si applica a formule di qualunque dimensione;
- AlphaZeroSAT: self-play e MCTS con una rete convoluzionale di politica e valore sulla matrice clausole $\times$ variabili, di dimensione fissa.

Che cosa ho fatto:

- ho portato Graph-Q-SAT da PyTorch 1.1 e AlphaZeroSAT da TensorFlow 1.x e GSL a un unico stack attuale PyTorch / PyTorch Geometric;
- GAT-Q-SAT, che aggiunge l'attenzione su grafo a Graph-Q-SAT, confrontato con esso a parità di training set e con due controlli;
- le euristiche ristrette di Shirokikh et al.

Piano: apprendimento sul grafo di una formula $\rightarrow$ Graph-Q-SAT e GAT-Q-SAT $\rightarrow$ AlphaZeroSAT $\rightarrow$ implementazione $\rightarrow$ risultati.

- La CNF come grafo
- Message Passing Neural Networks (MPNN)
- NeuroSAT, il primo modello su grafo per SAT

(ISPR – survey)

Gilmer et al., *Neural Message Passing for Quantum Chemistry*.
Selsam et al., *Learning a SAT Solver from Single-Bit Supervision*.

Background: apprendimento sul grafo di una CNF

La CNF come grafo

Una CNF è in modo naturale un grafo bipartito:

- un nodo per ogni variabile (o letterale) e per ogni clausola;
- un arco fra una variabile e ogni clausola in cui compare, con la polarità come feature dell'arco (nella vista per letterali, anche un arco fra x e $\bar{x}$).

La rappresentazione è invariante per permutazioni di variabili e clausole, e una sola rete si applica a formule di qualunque dimensione, cosa che la CNN a dimensione fissa non può fare. È la rappresentazione di NeuroSAT e di Graph-Q-SAT.

$$\varphi = (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_3)$$

variabili	clausole
x_1	
x_2	c_1
x_3	c_2

gli archi portano la polarità $\{+, -\}$ del letterale.

Message Passing Neural Networks (MPNN)

Uno schema generale per l'apprendimento su grafi (Gilmer et al., 2017). Ogni nodo v ha uno stato h_v^t, e per T round scambia messaggi con i suoi vicini $\mathcal{N}(v)$:

$$\text{messaggio:} \quad m_v^{t+1} = \sum_{w \in \mathcal{N}(v)} M_t(h_v^t, h_w^t, e_{vw})$$

$$\text{aggiornamento:} \quad h_v^{t+1} = U_t(h_v^t, m_v^{t+1})$$

$$\text{lettura:} \quad \hat{y} = R(\{\, h_v^T : v \in G \,\})$$

- M_t (messaggio), U_t (aggiornamento, spesso una GRU o una LSTM) e R (lettura) sono appresi e condivisi fra i nodi e fra i round.
- Le GNN con gate, le GCN, le GAT e la rete encode–process–decode sono tutte istanze di questo schema.

NeuroSAT: un solutore SAT da una supervisione di un bit

Embedding di letterali e clausole $L^{(t)} \in \mathbb{R}^{2n \times d}$, $C^{(t)} \in \mathbb{R}^{m \times d}$; adiacenza bipartita M; percettroni $L_{\text{msg}}, C_{\text{msg}}, L_{\text{vote}}$; aggiornamenti LSTM L_u, C_u. Un round:

$$(C^{(t+1)}, C_h^{(t+1)}) \leftarrow C_u([\, C_h^{(t)},\ M^\top L_{\text{msg}}(L^{(t)})\,])$$

$$(L^{(t+1)}, L_h^{(t+1)}) \leftarrow L_u([\, L_h^{(t)},\ F(L^{(t)}),\ M\, C_{\text{msg}}(C^{(t+1)})\,])$$

dopo T round, voto e media:

$$L_*^{(T)} \leftarrow L_{\text{vote}}(L^{(T)}), \qquad y^{(T)} \leftarrow \text{mean}\,(L_*^{(T)}).$$

F scambia ogni letterale con il suo complemento.

- Addestrato solo come classificatore di soddisfacibilità (un bit per formula).
- Eppure impara a trovare assegnamenti che soddisfano la formula, che compaiono nei voti dei letterali.
- Ha stabilito la vista a grafo di una CNF, su cui è costruito Graph-Q-SAT.
- Limite: un solutore a sé, non competitivo nel tempo con CDCL.

- Reinforcement learning per il branching
- Background
  - Deep Q-Learning (DQN)
  - Graph Networks
- Modello: encode–process–decode
- Attenzione su grafo (GAT) e due controlli

Graph-Q-SAT

&

GAT-Q-SAT

(ISPR + RL – survey con codice aggiuntivo)

Kurin et al., *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?*

Il branching come processo decisionale di Markov

Graph-Q-SAT trasforma la ricerca CDCL in un MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- stato s: la formula semplificata dall'assegnamento parziale, con le clausole imparate, come grafo bipartito;
- azione a: una variabile libera con un valore di verità, cioè una decisione;
- ricompensa $r = -0.1$ a ogni passo non terminale e 0 alla fine, quindi massimizzare il ritorno minimizza il numero di decisioni.

L'obiettivo è la funzione di valore delle azioni

$$Q^\star(s,a) = \mathbb{E}\big[\, r + \gamma \max_{a'} Q^\star(s',a') \,\big] \qquad \text{(Bellman)},$$

e l'euristica è $\pi(s) = \arg\max_a Q^\star(s,a)$ sui nodi variabile.

Deep Q-Learning (DQN)

$Q^\star$ è approssimata da una rete Q_θ, adattata minimizzando l'errore di differenza temporale su un replay buffer $\mathcal{D}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\Big(\underbrace{r + \gamma \max_{a'} Q_{\theta^-}(s', a')}_{\text{TD target}} - Q_\theta(s, a)\Big)^2\right]$$

- la target network θ^-, una copia di θ aggiornata ogni 10 passi, stabilizza il target;
- il replay buffer (20000 transizioni, campionate in modo uniforme) rompe la correlazione fra transizioni consecutive;
- esplorazione ε-greedy: un'azione a caso con probabilità ε, altrimenti $\arg\max_a Q_\theta$.

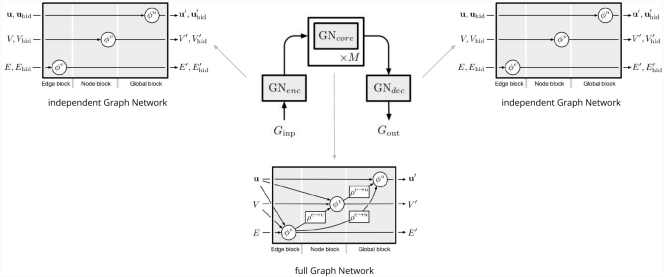
Il numero di azioni cambia lungo un episodio (due per variabile libera), e quindi Q_θ è una rete su grafo che dà un punteggio a ogni nodo variabile.

Modello: la rete encode--process--decode

Un blocco graph network (Battaglia et al.) aggiorna gli archi, poi i nodi, poi l'attributo globale:

$$e'_{ij} = \phi^e(e_{ij}, v_i, v_j, u)$$
$$v'_i = \phi^v(\sum_{j \in \mathcal{N}_i} e'_{ij}, v_i, u)$$
$$u' = \phi^u(\bar{e}', \bar{v}', u)$$

i ϕ sono percettroni; la somma è l'aggregazione ρ della figura. Encoder e decoder agiscono su ogni elemento in modo indipendente; il blocco centrale è applicato $M = 4$ volte con pesi condivisi, e il decoder dà due Q-value per variabile.



encode–process–decode: $\mathrm{GN}_{enc} \to \mathrm{GN}_{core} \times M \to \mathrm{GN}_{dec}$.

Configurazione e misura

La configurazione di Kurin et al.:

- feature dei nodi: one-hot a 2 dimensioni (variabile o clausola); feature degli archi: la polarità; l'attributo globale: una costante;
- embedding di dimensione 32 dall'encoder, percettroni con 64 unità nascoste nel blocco centrale, $M = 4$ round; 71144 parametri;
- Adam a $2 \cdot 10^{-5}$, $\gamma = 0.99$, minibatch di 64, 50000 aggiornamenti, al più 500 decisioni del modello per episodio (il cap).

La misura, MRIR:

$$\text{MRIR} = \text{median} \frac{\#\text{iter}_{\text{MiniSat}}}{\#\text{iter}_{\text{modello}}}$$

la Median Relative Iteration Reduction sulle istanze di una famiglia: sopra 1 il modello prende meno decisioni di MiniSat. Dopo il cap, VSIDS prende le decisioni rimanenti.

Idea di miglioramento: l'attenzione su grafo

- Nel blocco centrale un nodo somma i messaggi di tutti i suoi vicini con lo stesso peso; in una formula random le clausole sono statisticamente uguali, ma in una strutturata hanno spesso ruoli diversi (nel graph coloring, "almeno un colore" per un nodo e "colori diversi" per gli estremi di un arco).
- Idea: pesare i vicini con coefficienti appresi che dipendono dagli embedding degli archi, cioè una Graph Attention Network (GAT) dentro il blocco centrale.
- Nei termini di Battaglia et al., un blocco non locale inserito nel blocco graph network completo.

Veličković et al., *Graph Attention Networks* (ICLR 2018).

Graph Attention Networks (GAT)

Per un nodo i con vicini $\mathcal{N}_i$ e una mappa lineare condivisa W, l'attenzione pesa ogni messaggio con una rilevanza appresa:

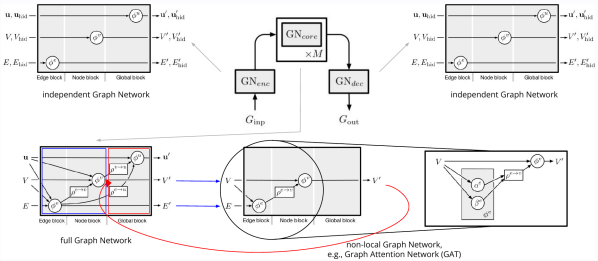
$$\begin{aligned} \text{score:} \quad & \omega_{ij} = \mathsf{LeakyReLU}(\mathbf{w}^\top [\, W v_i \,\|\, W v_j \,]) \\ \text{normalizzazione:} \quad & \alpha_{ij} = \frac{\exp \omega_{ij}}{\sum_{j' \in \mathcal{N}_i} \exp \omega_{ij'}} \\ \text{aggregazione:} \quad & v_i^+ = \sum_{j \in \mathcal{N}_i} \alpha_{ij} W v_j \end{aligned}$$

- multi-head: K attenzioni indipendenti, concatenate o mediate;
- con le feature degli archi: il punteggio legge anche l'embedding dell'arco, che porta la polarità del letterale.

GAT-Q-SAT e i due controlli

GAT-Q-SAT aggiunge due strati di attenzione con le feature degli archi (3 teste, poi 1) fra l'aggiornamento dei nodi e quello globale del blocco centrale, e tiene la somma: 113256 parametri, circa il 60% in più. Un guadagno può allora venire dall'attenzione o dalla capacità, e due controlli li separano:

- Graph-Q-SAT wide (104 unità nascoste, 113504 parametri, niente attenzione);
- attention aggregation (la somma sostituita da una media pesata dall'attenzione, 71627 parametri).



il blocco centrale con un blocco non locale, una GAT con attenzione sugli archi α^e e messaggi β^e.

- Reinforcement learning con ricerca per il branching
- Background
  - Monte Carlo Tree Search (MCTS)
  - Il self-play di AlphaZero
- Modello: una rete convoluzionale di politica e valore
- Risultati

(ISPR + RL – survey con codice)

Wang & Rompf, *From Gameplay to Symbolic Reasoning: Learning SAT Solver Heuristics in the Style of Alpha(Go) Zero.*

AlphaZeroSAT

Monte Carlo Tree Search (MCTS)

La risoluzione è trattata come un gioco: un nodo è un assegnamento parziale, una mossa un letterale di branching. Dalla radice si ripetono quattro passi:

1. selezione del figlio che massimizza il punteggio PUCT

$$a_t = \arg\max_a \Big(Q(s,a) + \underbrace{c_{\text{puct}}\, P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}}_{U(s,a)} \Big);$$

2. espansione di una foglia e sua valutazione con la rete $(P, v) = f_\theta(s)$;
3. bootstrap dal valore v;
4. propagazione all'indietro: $N \mathrel{+}= 1$, $W \mathrel{+}= v$, $Q = W/N$.

I conteggi delle visite danno una politica migliore, $\pi(a \mid s) \propto N(s,a)^{1/\tau}$.

AlphaZero: policy iteration con self-play

Una sola rete $(p, v) = f_\theta(s)$ dà una politica p sulle mosse e un valore $v \in [-1, 1]$ (quanto è probabile che la ricerca riesca da s).

- Self-play: MCTS, guidata da f_θ, dà una politica più forte π, giocata fino alla fine per ottenere un esito z.
- Apprendimento: v è regredito su z e p su π; MCTS migliora la politica e la rete la valuta:

$$\ell = \underbrace{(z - v)^2}_{valore} \quad \underbrace{- \pi^\top \log p}_{politica (cross-entropy)} \quad + \quad \underbrace{c\|\theta\|^2}_{regolarizzazione} .$$

- Niente dati umani e nessuna euristica scritta a mano: l'euristica si impara dal solo self-play.

AlphaZeroSAT: il modello

Stato: la CNF come matrice clausole $\times$ variabili di dimensione fissa (valori in $\{+1, 0, -1\}$ secondo la polarità, come canali di un'immagine).

Rete: un tronco convoluzionale ($32@8\times8$, $64@4\times4$, $64@3\times3$), poi due teste:

$$politica\ p = \text{softmax}(\text{logits} + (\text{mask} - 1)\cdot\infty) \in \Delta^{2n}$$
$$valore\ v = \tanh(\cdot) \in [-1, 1]$$

i letterali delle variabili assenti dalle clausole rimaste sono mascherati.

Model1/2/3 differiscono nelle teste (completamente connesse o convoluzionali), come nel progetto originale.

Ambiente: MiniSat esteso in un gioco per MCTS che supporta simulazioni ipotetiche.

Misura: il numero medio di decisioni di branching per risolvere (meno è meglio), come nell'articolo originale.

Risultato (`uf20-91`, 25 cicli di self-play): 6.70 decisioni in media su 64 problemi di test; la valutazione dopo ogni ciclo, su 16 problemi, va da 3.4 a 9.4, troppo rumorosa per mostrare una tendenza.

Ogni decisione è una ricerca MCTS, molto più lenta di una decisione di VSIDS.

Limite: con una matrice di dimensione fissa la rete non si applica ad altre dimensioni, ed è la ragione dei metodi su grafo.

- Il port
- Euristiche ristrette
- Una prova del port
- Dove l'attenzione aiuta

(il progetto di codice)

Implementazione & Risultati

Il port: un unico stack PyTorch attuale

- Graph-Q-SAT era scritto per PyTorch 1.1 con estensioni compilate a versione fissa (`torch-scatter`, `torch-sparse`); AlphaZeroSAT per TensorFlow 1.x, con GSL nel suo ambiente C++. Nessuno dei due si installa più.
- Graph-Q-SAT: le aggregazioni da `torch_geometric.utils.scatter`, l'ambiente costruito senza `gym.make`, il wrapper di MiniSat ricompilato su `numpy` 2, i checkpoint del 2021 caricati con il `GATConv` attuale.
- AlphaZeroSAT: la rete e la loss di AlphaZero riscritte in PyTorch, GSL tolta (il rumore di Dirichlet da `<random>` del C++11); si addestra da capo a fondo su GPU.
- Entrambi girano su Colab con i checkpoint su Drive; le istanze di Graph-Q-SAT stanno nel suo repository, quelle di AlphaZeroSAT in `data/`.

Euristiche ristrette: meno forward pass

Graph-Q-SAT riduce le iterazioni, ma un forward pass della rete a ogni decisione costa più di VSIDS. Tre modi di interrogare meno la rete (Shirokikh et al., 2023), in valutazione:

- rilascio anticipato (`--release_after T`): il modello prende le prime T decisioni, VSIDS le altre;
- pool di azioni (`--action_pool_size L`): un forward pass dà le L azioni migliori, eseguite in ordine;
- warm start (`--warmstart_release`): i Q-value della radice inizializzano le attività di VSIDS, $\text{activity}(x) = -1/\max\left(Q(x,\mathsf{T}), Q(x,\mathsf{F})\right)$, che poi lavora da sola.

Sulle istanze di questo progetto la ricerca è troppo breve perché il risparmio si veda, e non sono misurate.

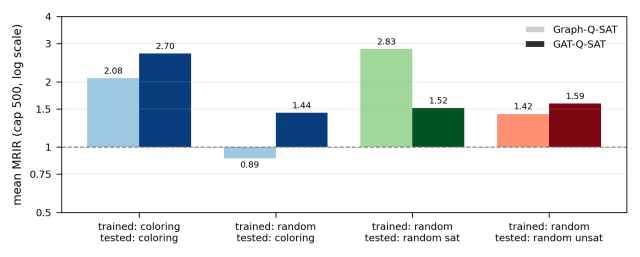
Una prova del port

- I checkpoint addestrati nel 2021 sullo stack originale, rivalutati sul nuovo, e confrontati con i log del 2021 istanza per istanza.
- In 63 valutazioni su 86 le iterazioni del modello coincidono su ogni istanza; nelle altre 23 la mediana cambia al più di 0.03:

`flat30-60`, cap 500	2021	2026
Graph-Q-SAT	1.833	1.833
GAT-Q-SAT	2.000	2.000

- Il training è stato verificato riaddestrando da zero i due modelli sul coloring con il nuovo stack.

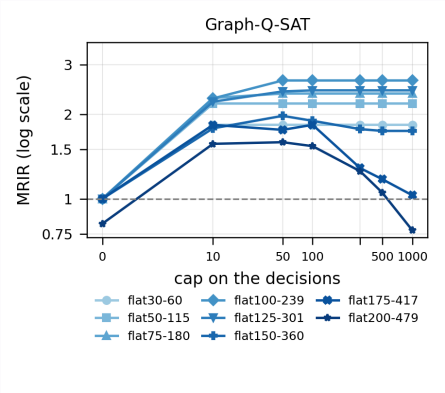
Risultati: Graph-Q-SAT e GAT-Q-SAT a parità di training set

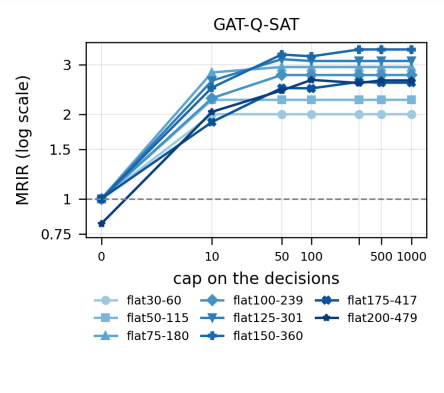


MRIR medio al cap 500; il colore è quello delle istanze di test, chiaro Graph-Q-SAT e scuro GAT-Q-SAT.

- Graph coloring: GAT-Q-SAT avanti, 2.70 contro 2.08.
- Addestrati su formule random, provati sul coloring: Graph-Q-SAT sotto MiniSat (0.89), GAT-Q-SAT sopra (1.44).
- Random 3-SAT soddisfacibile: Graph-Q-SAT avanti (2.83 contro 1.52).

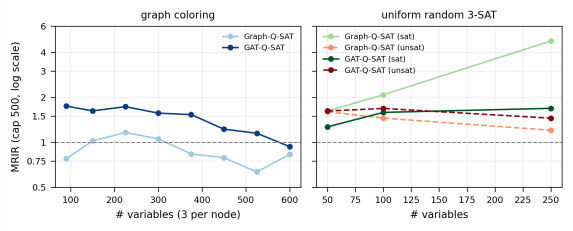
Risultati: l'MRIR in funzione del cap sul graph coloring





Una linea per famiglia, chiara la più piccola; sulle famiglie grandi le decisioni di Graph-Q-SAT dopo le prime decine allungano la ricerca.

Risultati: il divario cresce con la dimensione



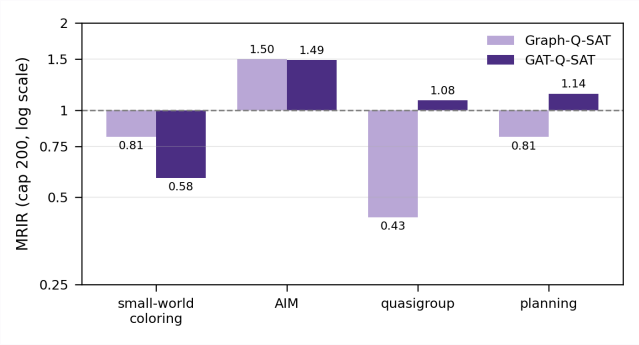
Addestrati su formule random: MRIR in funzione della dimensione (cap 500).

Addestrati e provati sul graph coloring, MRIR al cap 500:

famiglia	Graph	GAT	Δ
`flat30-60`	1.83	2.00	$+0.17$
`flat125-301`	2.43	3.09	$+0.66$
`flat150-360`	1.74	3.40	$+1.66$
`flat200-479`	1.05	2.64	$+1.59$

Graph-Q-SAT peggiora sulle istanze grandi, GAT-Q-SAT molto meno.

Risultati: un cambio di dominio, senza riaddestramento



I modelli addestrati sul coloring sulle tredici famiglie strutturate di SATLIB fino a 2000 variabili e 150000 clausole; cap 200, MRIR contro il MiniSat più forte.

[Su quante famiglie ogni modello è sopra MiniSat e sopra l'altro.]

Risultati: meno iterazioni, e il tempo

	cap 50		cap 500		cap 1000	
	sec	MRIR	sec	MRIR	sec	MRIR
Graph-Q-SAT	2.55	1.59	26.51	1.05	57.67	0.77
GAT-Q-SAT	3.52	2.43	12.20	2.64	12.19	2.64

`flat200-479`, secondi per istanza, mediana

Ogni decisione di un modello è un forward pass della rete: GAT-Q-SAT costa di più per decisione, ma ai cap più alti è il più veloce dei due, perché termina con meno decisioni. Entrambi impiegano secondi dove MiniSat impiega una frazione di secondo, ed è il punto delle euristiche ristrette.

Dove è andato il campo, e conclusioni

- NeuroSAT $\rightarrow$ NeuroCore $\rightarrow$ NeuroBack (ICLR 2024): un graph transformer interrogato una volta prima della ricerca, la cui inferenza si paga offline; la stessa direzione dell'attenzione di GAT-Q-SAT.
- L'attenzione su grafo aiuta sui problemi strutturati (graph coloring), con un divario che cresce con la dimensione, e anche quando il modello è addestrato su formule random; sul random 3-SAT uniforme soddisfacibile è migliore Graph-Q-SAT.
- [I controlli e il pannello di transfer, dalla nota.]
- La rete a dimensione fissa di AlphaZeroSAT non si applica ad altre dimensioni, ed è la ragione dei metodi su grafo.

Codice e slide: `github.com/dmeoli/NeuroSAT`; la nota su GAT-Q-SAT: `github.com/dmeoli/GQSAT`.

Grazie per l'*attenzione*!

github.com/dmeoli/NeuroSAT